

Encryption Method

1. Install Terraform on your system and verify the installation by running terraform --version in your terminal.

ANS

- I installed Terraform on my system. After installation, I opened the terminal and checked the installed version using:
 - terraform --version
 - ❖ **Expected Output:**
 - Terraform v1.x.x
 - This confirms that Terraform was installed successfully and is available from the terminal.
2. Write a basic Terraform script (main.tf) that provisions an AWS EC2 instance with the t2.micro type in the us-east-1 region.

Hint: Use the aws_instance resource and specify the AMI ID for Amazon Linux 2.

ANS

- I created a Terraform configuration file named main.tf to provision a t2.micro EC2 instance in the us-east-1 AWS region.

```
provider "aws" {  
    region = "us-east-1"  
}  
  
resource "aws_instance" "web_server" {  
    ami           = "ami-xxxxxxxxxxxxxxxxxxxxx"  
    instance_type = "t2.micro"  
}
```
 - Replace ami-xxxxxxxxxxxxxxxxxxxxx with the current **Amazon Linux 2 AMI ID for us-east-1** shown in your AWS Console. AMI IDs are region-specific.
3. Add a tag to your EC2 instance in your Terraform script so that it displays your name as the 'Owner' in the AWS console.

ANS

- I added an Owner tag to identify the owner of the EC2 instance in the AWS Console.

```

provider "aws" {
    region = "us-east-1"
}

resource "aws_instance" "web_server" {
    ami      = "ami-xxxxxxxxxxxxxxxxxxxx"
    instance_type = "t2.micro"
    tags = {
        Owner = "Deep Patel"
        Name  = "Terraform-Web-Server"
    }
}

```

- After the instance is created, the AWS Console displays:
 - Owner = Deep Patel
 - Name = Terraform-Web-Server

4. Initialize and apply your Terraform configuration to create the EC2 instance, then verify in the AWS console that your instance is running.

ANS

- I installed Terraform on my system. After installation, I opened the terminal and checked the installed version using:
 - terraform --version
- This confirms that Terraform was installed successfully and is available from the terminal.

1) Create an EC2 Instance using Terraform

- I created a Terraform configuration file named main.tf to provision a t2.micro EC2 instance in the us-east-1 AWS region.

```

provider "aws" {
    region = "us-east-1"
}

resource "aws_instance" "web_server" {
    ami      = "ami-xxxxxxxxxxxxxxxxxxxx"
    instance_type = "t2.micro"
}

```

2) Add Owner Tag

- I added an Owner tag to identify the owner of the EC2 instance in the AWS Console.

```
provider "aws" {  
    region = "us-east-1"  
}  
  
resource "aws_instance" "web_server" {  
    ami      = "ami-xxxxxxxxxxxxxxxxxxxxx"  
    instance_type = "t2.micro"  
    tags = {  
        Owner = "Deep Patel"  
        Name  = "Terraform-Web-Server"  
    }  
}
```

After the instance is created, the AWS Console displays:

Owner = Deep Patel

Name = Terraform-Web-Server

3) Initialize and Apply Terraform

- I opened the terminal in the directory containing main.tf and initialized Terraform.

Step 1: Initialize Terraform

- terraform init
- Terraform downloaded the required AWS provider and initialized the working directory.

Step 2: Check the configuration

- terraform plan
- This command shows the resources Terraform plans to create.

Step 3: Create the EC2 instance

- terraform apply
- I confirmed the operation by entering:
- Terraform then created the EC2 instance.

Step 4: Verify

- I opened the **AWS Management Console** → **EC2** → **Instances** and verified that the Terraform-created instance was in the:
- 5. Use ChatGPT or GitHub Copilot to help you write a Terraform script that creates a security group allowing inbound SSH (port 22) access, and add it to your EC2 instance.

Hint: Ask the AI tool for a minimal example with comments, then adapt it to your main.tf file.

ANS

- I used an AI tool to create a minimal Terraform configuration for an AWS security group that allows SSH access on port 22. I then added the security group to the EC2 instance.

❖ **Complete main.tf**

```
provider "aws" {  
    region = "us-east-1"  
}  
  
# Security Group  
resource "aws_security_group" "ssh_access" {  
    name      = "terraform-ssh-sg"  
    description = "Allow SSH access"  
    vpc_id    = "vpc-xxxxxxxxxxxxxxxxxxxx"  
    ingress {  
        description = "SSH"  
        from_port   = 22  
        to_port     = 22  
        protocol    = "tcp"  
        cidr_blocks = ["0.0.0.0/0"]  
    }  
    egress {  
        from_port = 0  
        to_port   = 0  
        protocol  = "-1"  
        cidr_blocks = ["0.0.0.0/0"]  
    }  
}
```

```

    }
    tags = {
        Name = "Terraform-SSH-SG"
        Owner = "Deep Patel"
    }
}
# EC2 Instance
resource "aws_instance" "web_server" {
    ami          = "ami-xxxxxxxxxxxxxxxxxx"
    instance_type = "t2.micro"
    vpc_security_group_ids = [
        aws_security_group.ssh_access.id
    ]
    tags = {
        Name = "Terraform-Web-Server"
        Owner = "Deep Patel"
    }
}

```

❖ **Final Commands**

- terraform init
- terraform plan
- terraform apply

- After successful execution, I verified the EC2 instance and its security group in the AWS Console.